

บทที่ 2

เว็บเซอร์วิส

เว็บเซอร์วิส คือ เว็บแอปพลิเคชันยุคใหม่ที่ประกอบด้วยส่วนย่อยๆ มีความสมบูรณ์ในตัวเอง สามารถติดตั้ง ค้นหา เริ่มทำงานได้ผ่านเว็บ เว็บเซอร์วิสสามารถทำอะไรก็ได้ตั้งแต่งานง่ายๆ เช่นดึงข้อมูล จนถึงกระบวนการทางธุรกิจที่ซับซ้อน เมื่อเว็บเซอร์วิสตัวใดตัวหนึ่งเริ่มเว็บเซอร์วิสตัวอื่นก็สามารถรับรู้ และเริ่มทำงานได้อีกด้วย

หลายคนอาจจะถามว่าทำไมต้องเป็นเว็บเพราะโดยปกติมีมิดเดิลแวร์อื่นๆ มากมายเช่น RMI Jini CORBA DCOM ฯลฯ แม้มีมิดเดิลแวร์เหล่านี้จะสามารถรองรับได้ แต่ไม่มีตัวใดตัวหนึ่งที่เด่นจริง แต่ในเมื่อเว็บมีจุดเด่นในเรื่องของการให้บริการข้อมูลที่สะดวก ใช้งานง่าย จึงกลายเป็นตัวประสานมิดเดิลแวร์ต่างๆ เข้าด้วยกันซึ่งจะช่วยให้คุยกันเองคงยากยิ่ง เว็บได้ทำหน้าที่เป็นตัวกลางให้ มิดเดิลแวร์เหล่านี้สามารถคุยกันได้และมีประสิทธิภาพกว่าวิธีการเดิมๆ มาก

หากเรามองจากกรณีของเอ็นทีเยอร์ (n-tier) แอปพลิเคชัน จะพบว่า เว็บเซอร์วิสคือกลไกในการเข้าถึงบริการที่แต่ละมิดเดิลแวร์ให้บริการ การเข้าถึงจะอาศัย Listener และส่วนประกอบที่ระบุถึงบริการต่างๆ ที่รองรับการทำงาน โดยการทำงานจริงๆ นั้นก็ใช้วิธีการปกติของ มิดเดิลแวร์ นั้นๆ พื้นฐานของเว็บเซอร์วิส คือ XML กับ HTTP ซึ่งจะพบว่า HTTP ก็เป็นที่รู้จักกันดี และไปได้ทั่วทุกแห่งที่มีอินเทอร์เน็ต ส่วน XML คือภาษาสากลที่สามารถปรับแต่งได้ตามใจชอบ เพื่อให้เกิดกิจกรรมระหว่างไคลเอนต์ และบริการ หรือระหว่างส่วนประกอบต่างๆ เบื้องหลังเว็บเซอร์วิสก็คือ ข้อความ XML จะถูกแปลงให้การขอบริการจากมิดเดิลแวร์ และผลที่ได้ก็จะแปลงกลับมาในรูป XML โดยมาตรฐาน XML นี้ทำให้เราไม่ต้องกังวลเรื่องของการเชื่อมโยงอีกต่อไป และโพรโตคอล ที่ส่งก็คือ HTTP นั่นเอง ถ้าท่านเชื่อมโยงกับ HTTP (หรือเว็บ) ได้ ท่านก็ใช้บริการทุกอย่างได้

แต่อย่างไรก็ตามการเข้าถึงและการส่งงานนั้นยังเป็นเพียงโครงสร้างพื้นฐาน แต่ในความเป็นจริงยังมีอะไรมากกว่านั้น เช่น การค้นหา การทำธุรกรรม ความปลอดภัย การพิสูจน์ตัวตน และอื่นๆ อันเป็นบริการที่ทำให้เป็นบริการพื้นฐานจริงๆ ระบบเพิ่มเติมที่ต้องมีและต้องรักษาความสะอาดและใช้งานง่ายไว้ด้วย พื้นฐานของเว็บเซอร์วิส เต็มรูปแบบคือ XML + HTTP + SOAP + WSDL + UDDI หรือในระดับสูงกว่านั้น แต่ไม่ได้ถือเป็นสิ่งจำเป็นเสมอไปก็ต้องเพิ่มเทคโนโลยี XAML, XLANG, XKMS, XFS เป็นต้น สำหรับรายละเอียดในบางหัวข้อมีดังต่อไปนี้คือ

2.1 XML คืออะไร

2.1.1 แนะนำ XML

XML ย่อมาจาก eXtensible Markup Language XML ถูกออกแบบมาเพื่อใช้อธิบายข้อมูล ใน XML จะไม่มีแท็ก (tag) ที่กำหนดไว้ล่วงหน้า เราจะต้องสร้างแท็กของเราขึ้นมาเอง และจะใช้ Document Type Definition (DTD) ในการอธิบายรูปแบบของข้อมูล

ข้อแตกต่างที่สำคัญระหว่าง XML และ HTML คือ XML ถูกออกแบบมาเพื่อใช้แลกเปลี่ยนข้อมูล ไม่ใช่มาแทนที่ HTML XML กับ HTML ถูกออกแบบมาเพื่อจุดประสงค์ที่ต่างกัน HTML จะเกี่ยวข้องกับการแสดงผลข้อมูล XML จะเกี่ยวข้องกับการอธิบายข้อมูล แท็กของ HTML เช่น <H1></H1> เป็นแท็กสำหรับบอกรูปแบบการแสดงผล ในขณะที่แท็กของ XML ต้องสร้างขึ้นมาเองเช่น

```
<BOOK>
  <NAME>Beginning XML</NAME>
  <ISDN>1-861003-41-2</ISDN>
  <PAGE>800</PAGE>
</BOOK>
```

จะใช้แสดงข้อมูลของหนังสือที่ประกอบด้วยอีลีเมนต์ (element) BOOK , NAME , ISDN และ PAGE ข้อมูลที่อยู่ในรูปแท็กนี้เป็นอิสระต่อซอฟต์แวร์และฮาร์ดแวร์ จึงเหมาะที่จะนำมาใช้ในการแลกเปลี่ยนข้อมูลระหว่างระบบบนอินเทอร์เน็ต การเปลี่ยนข้อมูลให้อยู่ในรูป XML เป็นการลดความซับซ้อน และสามารถสร้างข้อมูลที่ถูกอ่านโดยแอปพลิเคชันใด ๆ ก็ได้ นอกจากนั้นการเพิ่มเติมอีลีเมนต์จะไม่ทำให้แอปพลิเคชันเสียหาย เนื่องจากการอ่านข้อมูลโดย XML พาร์เซอร์ (Parser) จะใช้วิธีแยกข้อมูลจากอีลีเมนต์ที่ต้องการ จาก XML ข้างต้นนี้จะแยกข้อมูลอีลีเมนต์ NAME , ISDN และ PAGE หากเพิ่มอีลีเมนต์ AUTHOR เข้าไป แอปพลิเคชันก็ยังคงดึงข้อมูลในอีลีเมนต์เดิมโดยจะมองข้ามอีลีเมนต์ที่ไม่ได้กำหนดให้ แยกข้อมูลออกมาในตอนพัฒนาแอปพลิเคชัน

2.1.2 ไวยากรณ์ของ XML

กฎไวยากรณ์ของ XML นั้นง่ายและเคร่งครัดมาก กฎของมันง่ายต่อการเรียนรู้และง่ายต่อการใช้ ด้วยเหตุนี้การสร้างซอฟต์แวร์ในการอ่านและจัดการกับข้อมูลจึงเป็นเรื่องง่าย ตัวอย่างของเอกสาร XML ดังนี้

```
<?XML version="1.0"?>
<note>
<to>Tove</to>
```

```
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend!</body>
</note>
```

ในบรรทัดแรกคือการประกาศเอกสาร XML ที่บอกถึง XML เวอร์ชัน

บรรทัดต่อมาคือรูทอีลีเมนต์ (root element) ซึ่งคือ <note> อีก 4 บรรทัดถัดมาเป็นการอธิบายถึง 4 อีลีเมนต์ลูก (child element) ของรูท (to , from , heading , body) และบรรทัดสุดท้ายเป็นการกำหนดจุดสิ้นสุดของรูท

กฎของเอกสาร XML มีดังนี้

- **XML อีลีเมนต์ (XML element) ทุกตัวต้องมีการปิดแท็ก**

ใน HTML บางอีลีเมนต์ไม่จำเป็นต้องมีการปิดแท็ก แต่ใน XML ทุกอีลีเมนต์จำเป็นต้องมีการปิดแท็ก

<p>This is a paragraph	X
This is a paragraph	✓

- **XML แท็กมีความแตกต่างระหว่างตัวพิมพ์เล็กและตัวพิมพ์ใหญ่ (case sensitive)**

ใน XML แท็ก <Message> นั้นต่างจาก <message> เนื่องจาก XML นั้นมีคุณสมบัติที่แยกความแตกต่างระหว่างตัวพิมพ์เล็กและตัวพิมพ์ใหญ่

<Message>This is incorrect</message>	X
<message>This is correct</message>	✓

- **XML อีลีเมนต์ทุกตัวต้องมีการซ้อนกันอย่างเหมาะสม**

ใน HTML บางอีลีเมนต์สามารถเหลื่อมกันได้ แต่ใน XML นั้นต้องซ้อนกันเป็นชั้นโดยสมบูรณ์

<i>This text is bold and italic</i>	X
<i>This text is bold and italic</i>	✓

- **ทุกเอกสาร XML ต้องมีรูทแท็ก**

แท็กแรกในเอกสาร XML นั้นจะเป็นรูทแท็กและจะมีได้เพียงหนึ่งเดียวเท่านั้น อีลีเมนต์อื่น ๆ ทั้งหมดจะต้องถูกซ้อนอยู่ภายในรูทอีลีเมนต์เท่านั้น และอีลีเมนต์เหล่านั้นก็สามารถมีอีลีเมนต์ย่อย ได้อีกดังนี้

```

<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>

```

- ค่าของแอตทริบิวต์ (Attribute) ต้องมีอยู่ภายในเครื่องหมายคำพูด (“”)

XML อีลีเมนต์สามารถมีแอตทริบิวต์ได้ โดยที่ค่าของแอตทริบิวต์จะต้องอยู่ภายในเครื่องหมายคำพูด (“”)

เอกสาร XML ข้างล่างนี้ อันแรกผิด อันที่สองถูก

<note date=12/11/99>	X
<note date="12/11/99">	✓

2.1.3 XML อีลีเมนต์

XML อีลีเมนต์มีคุณสมบัติดังนี้

- XML อีลีเมนต์สามารถขยายได้ตามตัวอย่างนี้

```

<note>
  <to>Tove</to>
  <from>Jani</from>
  <body>Don't forget me this weekend!</body>
</note>

```

ถ้าเราจะสร้างแอปพลิเคชันที่สามารถแยกอีลีเมนต์ to , from และ body จาก เอกสาร XML เพื่อแสดงผลลัพธ์ดังนี้

MESSAGE

To:Tove

From: Jani

Don't forget me this weekend!

และ ถ้าเพิ่มข้อมูลเพิ่มเติมลงไปดังนี้

```
<note>
  <date>1999-08-01</date>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

เมื่อเอกสาร XML เป็นดังนี้แล้ว แอปพลิเคชันที่สร้างไว้จะไม่เสียหาย เนื่องจากแอปพลิเคชันจะยังคงค้นหาอิลีเมนต์ to , from และ body ในเอกสาร XML และยังคงให้ผลลัพธ์เหมือนเดิม

■ XML อิลีเมนต์มีความสัมพันธ์กัน

ในการที่จะเข้าใจถึงภาพรวมของ XML จะต้องเข้าใจถึงความสัมพันธ์ระหว่างชื่อของ XML อิลีเมนต์ที่ตั้งและคอนเทนต์ (content) ของอิลีเมนต์

คำอธิบายของหนังสือดังนี้

Book Title: My First XML

Chapter 1: Introduction to XML

What is HTML

What is XML

Chapter 2: XML Syntax

Elements must have a closing tag

Elements must be correctly nested

สามารถมีเอกสาร XML ที่อธิบายหนังสือนั้น ได้ดังนี้

```
<book>
  <title>My First XML</title>
  <prod id="33-657" media="paper"></prod>
  <chapter>Introduction to XML
    <para>What is HTML</para>
```

```

<para>What is XML</para>

</chapter>

<chapter>XML Syntax

  <para>Elements must have a closing tag</para>

  <para>Elements must be properly nested</para>

</chapter>

</book>

```

book เป็นรูทอีลีเมนต์ title และ chapter เป็นอีลีเมนต์ย่อยของ book book เป็นอีลีเมนต์หลักของทั้ง title และ chapter title และ chapter มีความเกี่ยวข้องกัน (sibling) เนื่องจากมีอีลีเมนต์หลักเดียวกัน

■ อีลีเมนต์มีคอนเทนต์

XML อีลีเมนต์คือทั้งหมดตั้งแต่เปิดแท็กจนถึงปิดแท็ก อีลีเมนต์หนึ่งสามารถมีอีลีเมนต์คอนเทนต์ (element content) , มิกซ์คอนเทนต์ (mixed content) , ซิมเปิลคอนเทนต์ (simple content) หรือ คอนเทนต์ว่างเปล่า (empty content) และยังมี แอตทริบิวต์ได้ด้วย

จากตัวอย่างข้างบน book มีอีลีเมนต์คอนเทนต์เพราะมันประกอบด้วยอีลีเมนต์อื่น chapter เป็นมิกซ์คอนเทนต์เพราะมันประกอบด้วยเท็กซ์และอีลีเมนต์อื่น ๆ para เป็นซิมเปิลคอนเทนต์ หรือเท็กซ์คอนเทนต์ (text content) เพราะมันประกอบด้วยเท็กซ์เท่านั้น prod เป็นเอ็มดีอีลีเมนต์เนื่องจากมันไม่มีข้อมูลใดๆ และเฉพาะ prod อีลีเมนต์เท่านั้นที่มีแอตทริบิวต์ แอตทริบิวต์ที่ชื่อ id มีค่าเท่ากับ “33-657” แอตทริบิวต์ที่ชื่อ media มีค่าเท่ากับ “paper”

■ การตั้งชื่ออีลีเมนต์

XML อีลีเมนต์มีกฎการตั้งชื่อดังนี้

- ชื่อประกอบด้วย ตัวอักษร , ตัวเลข และอักขระอื่นๆ
- ชื่อต้องไม่ขึ้นต้นด้วยตัวเลขหรือ “_” (underscore)
- ชื่อต้องไม่ขึ้นด้วยคำว่า XML (หรือ Xml หรือ xml)
- ชื่อไม่สามารถมีช่องว่างได้ (space)
- ชื่อต้องไม่ควรประกอบด้วย “:” เพราะมันถูกสงวนไว้กับการใช้เนมสเปซ (namespace)

ทุกๆชื่อสามารถใช้ได้ ไม่มีคำใดเป็นคำสงวน แต่จะเป็นการดีในการที่จะหลีกเลี่ยงการใช้ “-” หรือ “.” แล้วใช้เฉพาะ “_” เท่านั้น เนื่องจากอาจเกิดการสับสนของซอฟต์แวร์ในกรณีพยายามที่จะตัดคำ (first-name) หรือคิดว่า “name” เป็นคุณสมบัติของ object “first” (first.name)

2.1.4 XML แอตทริบิวต์

XML แอตทริบิวต์มีคุณสมบัติดังนี้

- รูปแบบของโค้ด (Quote) , “female” or ‘female’

ค่าของแอตทริบิวต์ต้องอยู่ในเครื่องหมายคำพูดเสมอ แต่อาจเป็น ‘ ซิงเกิ้ลโค้ด (single quote) หรือ “ อัญประกาศ(double quote) ก็ได้

```
<person sex="female">
```

✓

หรือ

```
<person sex='female'>
```

✓

อัญประกาศจะเป็นที่ใช้โดยทั่วไปมากกว่า แต่ในบางครั้ง (ถ้าค่าของแอตทริบิวต์จำเป็นที่จะต้องมีโค้ด) จึงจำเป็นที่จะต้องใช้อัญประกาศดังนี้

```
<gangster name='George "Shotgun" Ziegler'>
```

- การใช้อีลีเมนต์และแอตทริบิวต์

ข้อมูลสามารถที่จะถูกเก็บในอีลีเมนต์ย่อยหรือแอตทริบิวต์ ดังนี้

```
<person sex="female">
```

```
  <firstname>Anna</firstname>
```

```
  <lastname>Smith</lastname>
```

```
</person>
```

หรือ

```
<person>
```

```
  <sex>female</sex>
```

```
  <firstname>Anna</firstname>
```

```
  <lastname>Smith</lastname>
```

```
</person>
```

ตัวอย่างแรก sex เป็นแอตทริบิวต์ ตัวอย่างที่ 2 sex เป็นอิลีเมนต์ย่อยซึ่งทั้งสองตัวอย่างให้ความหมายเดียวกัน มันไม่มีกฎว่าจะใช้แอตทริบิวต์หรืออิลีเมนต์ย่อยเมื่อไร แต่ควรจะหลีกเลี่ยงการใช้แอตทริบิวต์ ควรใช้อิลีเมนต์ย่อย ถ้าอินฟอร์เมชันนั้นดูเหมือนจะเป็นข้อมูล (data)

■ หลีกเลี่ยงการใช้แอตทริบิวต์

บางปัญหาของการใช้แอตทริบิวต์

- แอตทริบิวต์ไม่สามารถมีหลายค่า (อิลีเมนต์ย่อยสามารถทำได้)
- แอตทริบิวต์ไม่ยากที่จะขยาย (ในอนาคต)
- แอตทริบิวต์ไม่สามารถอธิบายโครงสร้าง (อิลีเมนต์ย่อยสามารถทำได้)
- แอตทริบิวต์ยากที่จะจัดการด้วยโค้ดของโปรแกรม
- ค่าของแอตทริบิวต์ไม่ยากในการจะตรวจสอบกับ DTD ที่มีอยู่

แต่อย่างไรก็ตาม จะสามารถใช้แอตทริบิวต์ก็ต่อเมื่อ อินฟอร์เมชันนั้นไม่สัมพันธ์กับข้อมูล ตัวอย่างเช่น ในบางครั้ง ID อ้างอิง (reference) สามารถถูกใช้ในการเข้าถึง XML อิลีเมนต์

```
<messages>
  <note ID="501">
    <to>Tove</to>
    <from>Jani</from>
    <heading>Reminder</heading>
    <body>Don't forget me this weekend!</body>
  </note>
  <note ID="502">
    <to>Jani</to>
    <from>Tove</from>
    <heading>Re: Reminder</heading>
    <body>I will not!</body>
  </note>
</messages>
```

ID ในตัวอย่างนี้เป็นตัวนับ (counter) หรือค่าที่ไม่ซ้ำกัน (unique identifier) ในการอ้างอิง note ที่ต่างกันของไฟล์ XML ไม่ใช่ส่วนหนึ่งของข้อมูล note

2.1.5 ความถูกต้อง XML (XML Validation)

เอกสาร XML แบ่งการตรวจสอบความถูกต้องของเอกสารเป็น 2 ชนิดคือ

1) “Well Formed” XML Document

“Well Formed” XML document เป็นเอกสารที่เป็นไปตามกฎไวยากรณ์ XML ที่ได้กล่าวมาข้างต้น

2) “Valid” XML Document

“Valid” XML document เป็นเอกสาร XML ที่มีรูปแบบที่เป็นไปตามกฎของ DTD โดยจะต้องตรงตามโครงสร้างข้อมูลของเอกสารที่ได้ประกาศโดย DTD

■ XML DTD

วัตถุประสงค์ของ DTD คือกำหนดโครงสร้างของเอกสาร XML เพื่อใช้ในการตรวจสอบว่าเอกสาร XML นั้นถูกต้องตามรูปแบบหรือไม่ เช่น element BOOK จะต้องมีการมี element NAME เสมอ อาจจะไม่มี element PAGE หรือไม่ก็ได้ เป็นต้น

■ โครงสร้างของ XML (XML Schema)

โครงสร้างของ XML มีวัตถุประสงค์เหมือนกับ DTD จะถูกนำมาใช้แทน DTD เนื่องจาก

- ง่ายในการเรียนรู้มากกว่า DTD
- ใช้ได้ดีกว่า DTD
- โครงสร้าง (schema) เขียนด้วย XML
- รองรับค่าไทป์ (datatype)
- รองรับนามสเปซ

2.1.6 แนะนำ DTD

1) การประกาศ DOCTYPE ภายใน

ถ้า DTD ถูกรวมอยู่ในไฟล์ XML นั้น มันจะต้องอยู่ภายในการให้คำจำกัดความ DOCTYPE ตามไวยากรณ์นี้

```
<!DOCTYPE root-element [element-declarations]>
```

ตัวอย่างดังนี้

```
<?XML version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
```

```

<!ELEMENT from (#PCDATA)>

<!ELEMENT heading (#PCDATA)>

<!ELEMENT body (#PCDATA)>

]>

<note>

  <to>Tove</to>

  <from>Jani</from>

  <heading>Reminder</heading>

  <body>Don't forget me this weekend</body>

</note>

```

DTD ข้างบนนี้สามารถแปลความหมายได้ดังนี้

!DOCTYPE note (บรรทัดที่ 2) กำหนดว่าเป็นเอกสารของไทป์ (type) note

!ELEMENT note (บรรทัดที่ 3) กำหนดว่า note element มี 4 อีลิเมนต์ คือ to , from , heading และ body

!ELEMENT to (บรรทัดที่ 4) กำหนดว่า to อีลิเมนต์เป็นข้อมูลชนิด #PCDATA

!ELEMENT from (บรรทัดที่ 5) กำหนดว่า from อีลิเมนต์เป็นข้อมูลชนิด #PCDATA

2) การประกาศ DOCTYPE ภายนอก

ถ้า DTD อยู่ภายนอกไฟล์ XML มันจะต้องอยู่ภายในการให้คำจำกัดความ DOCTYPE ตามไวยากรณ์นี้

```
<!DOCTYPE root-element SYSTEM "filename">
```

ตัวอย่างไฟล์ XML เป็นดังนี้

```

<?XML version="1.0"?>

<!DOCTYPE note SYSTEM "note.dtd">

<note>

  <to>Tove</to>

  <from>Jani</from>

  <heading>Reminder</heading>

  <body>Don't forget me this weekend!</body>

</note>

```

ไฟล์ของ DTD ชื่อ note.dtd เป็นดังนี้

note.dtd

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

2.1.6.1 โครงสร้างของ XML

เอกสาร XML จะถูกสร้างจากส่วนต่างๆ ดังนี้

- อีลีเมนต์ – เป็นโครงสร้างหลักของ XML
- แท็ก - ใช้ในการกำหนดมาร์กอัปอีลีเมนต์ (markup element)
- แอตทริบิวต์ - ให้ข้อมูลเพิ่มเติมกับอีลีเมนต์
- เอ็นติตี้ (entity) – เป็นตัวแปรที่ใช้ในการกำหนดเท็กซ์ทั่ว ๆ ไป เอ็นติตี้อ้างอิง (entity reference) เป็นการอ้างอิงเอ็นติตี้ เช่น เอ็นติตี้ชื่อ domain มีค่าเท่ากับ www.w3.org entity reference คือ &domain หากภายในเอกสาร XML มี &domain ค่าของมันจะมีค่าเท่ากับ www.w3.org
- PCDATA -หมายถึง parse character data PCDATA เป็นเท็กซ์ที่จะถูกแปล (parse) โดยพาร์เซอร์
- CDATA –หมายถึง character data เป็นเท็กซ์ที่จะไม่ถูกแปลโดยพาร์เซอร์

2.1.6.2 การกำหนดอีลีเมนต์ใน DTD

วิธีการในการประกาศอีลีเมนต์จะเป็นดังนี้

- การประกาศอีลีเมนต์
ประกาศตามไวยากรณ์นี้

```
<!ELEMENT element-name category>
```

หรือ

```
<!ELEMENT element-name (element-content)>
```

- อีลีเมนต์ว่างเปล่า (Empty)
ถูกประกาศโดยคีย์เวิร์ด EMPTY

```
<!ELEMENT element-name EMPTY>
```

DTD example:

```
<!ELEMENT br EMPTY>
```

XML example:

```
<br />
```

- อีลีเมนต์ที่เป็นข้อมูลชนิดตัวอักษร

อีลีเมนต์ที่เป็นตัวอักษรเท่านั้นจะถูกประกาศด้วยคีย์เวิร์ด PCDATA ที่เปิดและปิดด้วย “(” “)”

```
<!ELEMENT element-name (#PCDATA)>
```

DTD example:

```
<!ELEMENT note (#PCDATA)>
```

- อีลีเมนต์ที่เป็นข้อมูลใด ๆ

ประกาศกับคีย์เวิร์ด ANY คอนเท้นต์จะสามารถเป็นข้อมูลชนิดใดก็ได้

```
<!ELEMENT element-name ANY>
```

DTD example:

```
<!ELEMENT note ANY>
```

- อีลีเมนต์และอีลีเมนต์ลูก (Sequences)

ประกาศโดยใช้ “,” ในการแยกอีลีเมนต์ลูกแล้วเปิดและปิดด้วย “(” “)”

```
<!ELEMENT element-name (child-element-name)>
```

หรือ

```
<!ELEMENT element-name (child-element-name,child-element-name,.....)>
```

DTD example:

```
<!ELEMENT note (to,from,heading,body)>
```

ถ้ามีการประกาศอีลีเมนต์ลูกจะต้องประกาศอีลีเมนต์ลูกให้สมบูรณ์ด้วยดังตัวอย่างข้างล่างนี้

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to    (#PCDATA)>
<!ELEMENT from  (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body  (#PCDATA)>
```

- การประกาศเพียงครั้งเดียวของอิลีเมนต์เดียวกัน

```
<!ELEMENT element-name (child-name)>
```

DTD example:

```
<!ELEMENT note (message)>
```

จากตัวอย่างหมายความว่าอิลีเมนต์ note จะประกอบด้วยอิลีเมนต์ลูก message 1 ค่าเท่านั้น

- การประกาศอย่างน้อย 1 ครั้งของอิลีเมนต์เดียวกัน

```
<!ELEMENT element-name (child-name+)>
```

DTD example:

```
<!ELEMENT note (message+)>
```

เครื่องหมาย + ถูกใช้ในความหมายหนึ่งหรือมากกว่า จึงหมายความว่าอิลีเมนต์ note จะประกอบด้วยอิลีเมนต์ลูก message ตั้งแต่ 1 ค่าขึ้นไป

- การไม่ประกาศหรือประกาศเท่าไรก็ได้ของอิลีเมนต์เดียวกัน

```
<!ELEMENT element-name (child-name*)>
```

DTD example:

```
<!ELEMENT note (message*)>
```

เครื่องหมาย * ถูกใช้ในความหมายว่าไม่มีหรือมีเท่าไรก็ได้ จึงหมายความว่าอิลีเมนต์ note จะประกอบด้วยอิลีเมนต์ลูก message ก็ค่าก็ได้ หรือไม่มีเลยก็ได้

- การไม่ประกาศหรือประกาศเพียงครั้งเดียวของอีลีเมนต์เดียวกัน

```
<!ELEMENT element-name (child-name?)>
```

DTD example:

```
<!ELEMENT note (message?)>
```

เครื่องหมาย ? ถูกใช้ในความหมายไม่มีหรือมีเพียงหนึ่ง จึงหมายความว่าอีลีเมนต์ note จะประกอบด้วยอีลีเมนต์ลูก message หนึ่งค่า หรือ ไม่มีก็ได้

- การประกาศคอนเทนต์แบบให้เลือก

จะใช้เครื่องหมาย | แทนความหมายหรือ

DTD example:

```
<!ELEMENT note (to,from,header,message|body)>
```

ดังนั้นตัวอย่างนี้จึงหมายถึงอีลีเมนต์ note จะต้องประกอบด้วยอีลีเมนต์ to, อีลีเมนต์ from, อีลีเมนต์ header และอีลีเมนต์ message หรืออีลีเมนต์ body ใดอย่างหนึ่งเท่านั้น

- การประกาศคอนเทนต์แบบผสม

เป็นการผสมการประกาศหลาย ๆ อย่างเข้าด้วยกัน

DTD example:

```
<!ELEMENT note (#PCDATA|to|from|header|message)*>
```

ตัวอย่างนี้หมายถึงอีลีเมนต์ note จะประกอบด้วยตัวอักษรที่ไม่ถูกแปลโดยพาร์เซอร์ หรืออีลีเมนต์ to, from, header, และ message จำนวนเท่าไรก็ได้

2.1.6.3 การกำหนดแอตทริบิวต์ใน DTD

วิธีการประกาศแอตทริบิวต์จะเป็นดังนี้

- การประกาศแอตทริบิวต์

แอตทริบิวต์ต้องถูกประกาศตามไวยากรณ์ นี้

```
<!ATTLIST element-name attribute-name attribute-type default-value>
```

DTD example:

```
<!ATTLIST payment type CDATA "check">
```

XML example:

```
<payment type="check">
```

ชนิดของแอตทริบิวต์สามารถมีค่าได้ดังตารางที่ 2-1

ค่า	คำอธิบาย
CDATA	ค่าต้องเป็นตัวอักษรที่จะไม่ถูกแปลโดยพาร์เซอร์
(en1 en2 ..)	ค่าต้องเป็นอันใดอันหนึ่งในรายการนั้น
ID	ค่าต้องเป็น id ที่ไม่ซ้ำกัน
IDREF	ค่าต้องเป็น id ของอีลิเมนต์อื่น
IDREFS	ค่าต้องเป็นรายการของ id อื่น โดยจะเว้นแต่ละรายการด้วยช่องว่าง
NMTOKEN	ค่าต้องเป็นชื่อที่ประกอบด้วยตัวอักษร , ตัวเลข , เครื่องหมายจุด (.) , เครื่องหมายขีดกลาง (-) , เครื่องหมายขีดล่าง (_) และสามารถมีเครื่องหมายโคลอน (:) ยกเว้นในตำแหน่งแรกได้
NMTOKENS	ค่าต้องเป็นรายการของ NMTOKEN โดยจะเว้นแต่ละรายการด้วยช่องว่าง
ENTITY	ค่าต้องเป็นเอนทิตี
ENTITIES	ค่าต้องเป็นรายการของเอนทิตี
NOTATION	ค่าต้องเป็นชื่อของ notation (notation คือชนิดของข้อมูลภายนอกที่กำหนดขึ้นเอง เช่น GIF เพื่อบอกว่าเป็นไฟล์รูปภาพ)
XML:	ค่าต้องขึ้นต้นด้วย XML

ตารางที่ 2-1 ชนิดของ แอตทริบิวต์ใน XML

ค่าดีฟอลต์ (default-value) สามารถมีค่าได้ดังตารางที่ 2-2

ค่า	คำอธิบาย
#DEFAULT value	ค่าดีฟอลต์ของแอตทริบิวต์
#REQUIRED	อีลิเมนต์ต้องมีแอตทริบิวต์

#IMPLIED	อิลีเมนต์จะมีแอตทริบิวต์หรือไม่ก็ได้
#FIXED value	แอตทริบิวต์ต้องเหมือนกับที่กำหนดไว้

ตารางที่ 2-2 ค่าดีฟอลต์ของ แอตทริบิวต์ใน XML

- ตัวอย่างของการประกาศ แอตทริบิวต์

DTD example:

```
<!ELEMENT square EMPTY>
```

```
<!ATTLIST square width CDATA "0">
```

XML example:

```
<square width="100"></square>
```

ตัวอย่างนี้หมายความว่าสแควร์อิลีเมนต์ (square element) เป็นอิลีเมนต์ว่างกับแอตทริบิวต์ width ซึ่งเป็นชนิดซิงเกิลค่า ถ้าไม่ระบุจะยึดค่าดีฟอลต์เป็น 0

- ค่าของ แอตทริบิวต์แบบดีฟอลต์

```
<!ATTLIST element-name attribute-name attribute-type "default-value">
```

DTD example:

```
<!ATTLIST payment type CDATA "check">
```

XML example:

```
<payment type="check">
```

ถ้าในเอกสาร XML ไม่มีการระบุจะใช้ค่าดีฟอลต์ แต่ถ้าระบุก็จะใช้ตามที่ระบุไว้

- แอตทริบิวต์โดยนัย

```
<!ATTLIST element-name attribute-name attribute-type #IMPLIED>
```

DTD example:

```
<!ATTLIST contact fax CDATA #IMPLIED>
```

XML example:

```
<contact fax="555-667788">
```


จะใช้ #IMPLIED ในกรณีจะระบุแอตทริบิวต์หรือไม่ก็ได้ ถ้าไม่ระบุก็จะมีค่าดีฟอลต์ให้

- แอตทริบิวต์ที่ต้องระบุเสมอ

```
<!ATTLIST element-name attribute_name attribute-type #REQUIRED>
```

DTD example:

```
<!ATTLIST person number CDATA #REQUIRED>
```

XML example:

```
<person number="5677">
```

ใช้ในการบังคับให้ต้องระบุแอตทริบิวต์ถ้าในเอกสาร XML ไม่ระบุ แอตทริบิวต์เอกสารนั้นจะไม่ถูกต้อง

- ค่าของแอตทริบิวต์คงที่

```
<!ATTLIST element-name attribute-name attribute-type #FIXED "value">
```

DTD example:

```
<!ATTLIST sender company CDATA #FIXED "Microsoft">
```

XML example:

```
<sender company="Microsoft">
```

ในแบบนี้ถ้าไม่ได้ระบุค่าแอตทริบิวต์จะใช้ค่าดีฟอลต์ แต่ถ้าจะระบุต้องระบุให้ตรงกับค่าดีฟอลต์ มิฉะนั้นจะถือว่าผิด

- ค่าของแอตทริบิวต์กำหนดเอง

```
<!ATTLIST element-name attribute-name (en1|en2|..) default-value>
```

DTD example:

```
<!ATTLIST payment type (check|cash) "cash">
```

XML example:

```
<payment type="check">
```

หรือ

```
<payment type="cash">
```

จะใช้เมื่อเราต้องการกำหนดค่าของแอตทริบิวต์เอง

2.1.6.4 การกำหนดเอนทิตีใน DTD

- การประกาศเอนทิตีภายใน

```
<!ENTITY entity-name "entity-value">
```

DTD Example:

```
<!ENTITY writer "Donald Duck.">
```

```
<!ENTITY copyright "Copyright W3Schools.">
```

XML example:

```
<author>&writer;&copyright;</author>
```

- การประกาศเอนทิตีภายนอก

```
<!ENTITY entity-name SYSTEM "URI/URL">
```

DTD Example:

```
<!ENTITY writer
```

```
SYSTEM "http://www.w3schools.com/entities/entities.XML">
```

```
<!ENTITY copyright
```

```
SYSTEM "http://www.w3schools.com/entities/entities.dtd">
```

XML example:

```
<author>&writer;&copyright;</author>
```

2.1.7 XML เนมสเปซ

XML เนมสเปซจะหาวิธีหลีกเลี่ยงความสับสนที่เกิดจากชื่ออิลิเมนต์ (Name Conflicts)

- ความสับสนที่เกิดจากชื่อ

ความสับสนที่เกิดจากชื่ออิลิเมนต์จะเกิดขึ้นบ่อยเมื่อเอกสารที่แตกต่างกันใช้ชื่อเดียวกันในการอธิบายชนิดของอิลิเมนต์ที่แตกต่างกัน เช่น ตารางใน 2 ตัวอย่างต่อไปนี้

```
<table>
```

```
<tr>
<td>Apples</td>
<td>Bananas</td>
</tr>
</table>
```

และ

```
<table>
<name>African Coffee Table</name>
<width>80</width>
<length>120</length>
</table>
```

ถ้านำเอกสาร XML ทั้ง 2 มารวมกัน จะเกิดความสับสนจากชื่อขึ้นเพราะทั้ง 2 มีอีลีเมนต์ (table) ซึ่งมีใช้ในความหมายที่ต่างกัน

- การแก้ปัญหความสับสนที่เกิดจากชื่อโดยใช้คำเติมหน้า (Prefix)

เอกสาร XML ทั้ง 2 คือ

```
<h:table>
<h:tr>
<h:td>Apples</h:td>
<h:td>Bananas</h:td>
</h:tr>
</h:table>
```

และ

```
<f:table>
<f:name>African Coffee Table</f:name>
<f:width>80</f:width>
<f:length>120</f:length>
</f:table>
```

ความสับสนที่เกิดจากชื่อจะถูกกำจัดไปโดยการเพิ่มคำเติมหน้าทำให้ได้อีลีเมนต์ที่ต่างกัน 2 ชนิด (<h:table> และ <f:table>)

■ การใช้เนมสเปซ

จากเอกสาร XML ทั้ง 2 คือ

```
<h:table xmlns:h="http://www.w3.org/TR/html4/">
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>
```

และ

```
<f:table xmlns:f="http://www.w3schools.com/furniture">
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

เราสามารถเพิ่มแอตทริบิวต์ xmlns ลงในแท็ก เพื่อบอกชื่อที่ระบุไว้ ที่เกี่ยวข้องกับเนมสเปซ

■ แอตทริบิวต์เนมสเปซ

แอตทริบิวต์เนมสเปซจะวางในแท็กเริ่มต้น (start tag) ตามไวยากรณ์ ดังนี้

```
Xmlns:namespace-prefix="namespace"
```

จากตัวอย่างข้างบนเนมสเปซจะสามารถกำหนดด้วยที่อยู่ในอินเทอร์เน็ต (Internet address)

```
Xmlns:f="http://www.w3schools.com/furniture">
```

จากเนมสเปซเจาะจง (namespace specification) ของ W3C กล่าวไว้ว่าเนมสเปซสามารถเป็น Uniform Resource Identifier (URI)

ถ้าเนมสเปซถูกกำหนดในแท็กเริ่มต้นแล้วอีลีเมนต์ลูกทุกตัวที่มีคำเต็มหน้าเหมือนกันจะต้อง มีความสัมพันธ์กับเนมสเปซเดียวกันด้วย

และที่อยู่ที่ใช้บอกเนมสเปซใช้เพื่อให้ชื่อมีความเป็นเอกลักษณ์ไม่ได้ใช้เป็นที่อยู่เพื่ออ้างหาข้อมูล แต่หลายบริษัทใช้เนมสเปซเพื่อเป็นตัวชี้ไปยังหน้าเว็บที่มีอยู่จริงซึ่งเก็บข้อมูลเกี่ยวกับเนมสเปซไว้

- **ดีฟอลต์เนมสเปซ (Default Namespaces)**

การกำหนดดีฟอลต์เนมสเปซจะป้องกันการใช้คำเต็มหน้าในอีลีเมนต์ลูกทุกตัว มีไวยากรณ์ดังนี้

```
<element xmlns="namespace">
```

ซึ่งจากตัวอย่างที่ผ่านมาจะได้ดังนี้

```
<table xmlns="http://www.w3.org/TR/html4/">
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

และ

```
<table xmlns="http://www.w3schools.com/furniture">
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

2.1.8 XML PCDATA และ CDATA

Parsed Character Data (PCDATA) คือข้อมูลแบบเท็กซ์ที่จะถูกแปลโดยพาร์เซอร์

Character Data (CDATA) คือข้อมูลแบบเท็กซ์ที่ไม่ถูก parse โดยพาร์เซอร์

2.1.8.1 PCDATA

- **XML พาร์เซอร์จะมองทุกเท็กซ์เป็น Parsed Characters (PCDATA)**

เมื่อ XML ถูกแปลเท็กซ์ที่อยู่ระหว่าง XML แท็กจะถูกแปลไปด้วย

```
<message>This text is also parsed</message>
```

พาร์เซอร์ทำเช่นนั้นเนื่องจาก XML อีลีเมนต์สามารถมีหลายอีลีเมนต์อยู่ภายใน ดังในตัวอย่าง

```
<name><first>Bill</first><last>Gates</last></name>
```

และพาร์เซอร์จะแบ่งเป็นซับอีลีเมนต์ (sub-element) ดังนี้

```
<name>
  <first>Bill</first>
  <last>Gates</last>
</name>
```

■ ตัวอักษรเอสเคป (Escape Character)

ตัวอักษร XML ที่ไม่สามารถใช้ได้จะถูกแทนที่ด้วยเอนติตี้อ้างอิง

ถ้าใส่ตัวอักษร “<” ใน XML อีลีเมนต์จะทำให้เกิดความผิดพลาด เพราะพาร์เซอร์จะแปลเป็นจุดเริ่มต้นของอีลีเมนต์ใหม่ ดังนั้นจึงไม่สามารถเขียน อย่างนี้ได้

```
<message>if salary < 1000 then</message>
```

การหลีกเลี่ยง สามารถทำได้โดย แทนที่ “<” ด้วยเอนติตี้อ้างอิงดังนี้

```
<message>if salary &lt; 1000 then</message>
```

เอนติตี้อ้างอิงมาตรฐานที่ได้กำหนดมาให้ก่อนแล้วใน XML มีทั้งหมด 5 ตัว ดังตารางที่ 2-3

Entity Reference	ค่า	ความหมาย
<	<	น้อยกว่า
>	>	มากกว่า
&	&	ampersand
'	'	apostrophe
"	"	quotation mark

ตารางที่ 2-3 เอนติตี้อ้างอิงมาตรฐาน

เอนติตี้อ้างอิงจะขึ้นต้นด้วย & และลงท้ายด้วย ;

เฉพาะ “<” และ “&” เท่านั้นที่ไม่สามารถใช้ได้ใน XML ส่วนตัวที่เหลือควรทำให้เป็นนิสย

2.1.8.2 CDATA

พาร์เซอร์จะไม่แปลทุกอย่างที่อยู่ในส่วนของ CDATA

ถ้าเท็กซ์มี “<” หรือ “&” อยู่ภายใน XML element นั้นจะถูกกำหนดเป็นส่วนของ CDATA ส่วนของ CDATA จะเริ่มต้นด้วย “<![CDATA[“ และลงท้ายด้วย “]]>”

```
<script>
<![CDATA[
function matchwo(a,b)
{
if (a < b && a < 0) then
{
return 1
}
else
{
return 0
}
}
]]>
</script>
```

จากตัวอย่างข้างต้น ทุกอย่างที่อยู่ในส่วนของ CDATA จะถูกเพิกเฉยโดยพาร์เซอร์

2.1.9 XML พาร์เซอร์

XML พาร์เซอร์คือโปรแกรมที่ใช้ในการอ่าน สร้าง และจัดการกับเอกสาร XML รวมถึงตรวจสอบความถูกต้องของเอกสาร XML พาร์เซอร์สามารถเขียนได้ด้วยภาษาใด ๆ ก็ได้ ในบราวเซอร์ IE 5.0 ขึ้นไปก็มี Microsoft XML parser ทำให้ IE สามารถแสดงผลเอกสาร XML ได้ XML พาร์เซอร์ของภาษาจาวาอย่างเช่น XML พาร์เซอร์ของซัน (<http://java.sun.com/xml>) หากเราต้องการสร้างแอปพลิเคชันที่สามารถจัดการกับ XML เราก็ต้องมีพาร์เซอร์ของภาษานั้นและเขียนโค้ดติดต่อกับ API ของมัน

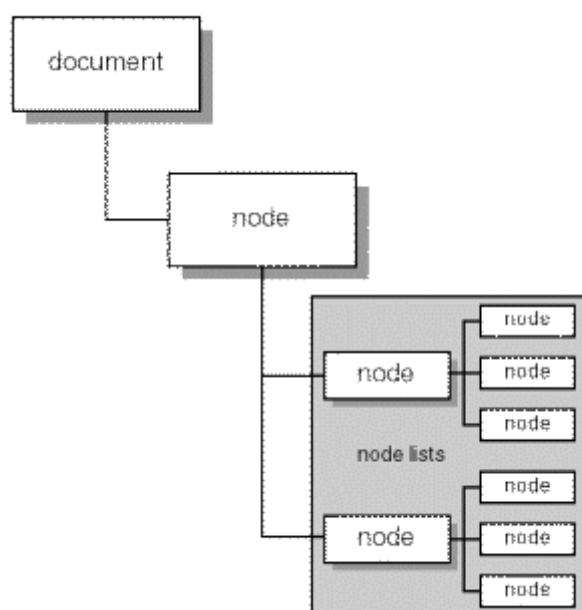
XML พาร์เซอร์มี 2 ชนิดคือ

- DOM (Document Object Model)
- SAX (Simple API for XML)

2.1.9.1 DOM

DOM จะใช้วิธีแปลเอกสาร XML เป็นลำดับชั้น (hierachy) ของออบเจ็กต์ดังนี้ (ดังรูปที่ 2-1)

- 1) เอกสารอ็อบเจ็กต์ (Document Object) – แหล่งข้อมูล XML
- 2) โหนดอ็อบเจ็กต์ (Node Object) – โหนดพ่อ(Parent node) ของโหนดลูก (child node)
- 3) โหนดลิสต์อ็อบเจ็กต์ (Node List Object) – ลิสต์ของ sibling node
- 4) แปลผิดพลาด (Parse Error) – ใช้ในการรับค่าผิดพลาดที่เกิดจากการแปล



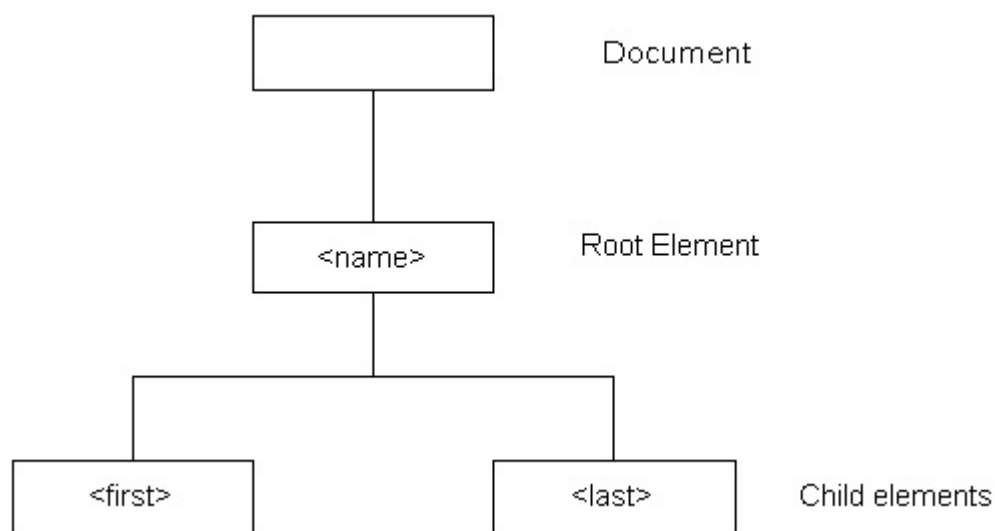
รูปที่ 2-1 โครงสร้างของ DOM

หากเอกสาร XML เป็นดังนี้

```

<name>
  <first>Jim(</first>
  <last>Jones</last>
</name>
  
```

จะได้ความสัมพันธ์ดังรูปที่ 2-2



รูปที่ 2-2 ความสัมพันธ์ของการแปลด้วย DOM

2.1.9.2 SAX

SAX (Simple API for XML) วิธีนี้จะเป็นแบบตอบสนองเหตุการณ์ event-driven โดยเหตุการณ์จะประกอบด้วย

- 1) อีลีเมนต์เริ่มต้น (Start Element) – จะตอบสนองกับเหตุการณ์นี้เมื่อพบจุดเริ่มต้นของแท็ก XML
- 2) อีลีเมนต์สิ้นสุด (End element) – จะตอบสนองกับเหตุการณ์นี้เมื่อพบจุดสิ้นสุดของแท็ก
- 3) ตัวอักษร (Character) – จะตอบสนองกับเหตุการณ์นี้เมื่อพบกับตัวอักษร
- 4) เริ่มต้นเอกสาร (Start Document) – จะตอบสนองกับเหตุการณ์นี้ในตอนเริ่มแปล
- 5) สิ้นสุดเอกสาร (End Document) - จะตอบสนองกับเหตุการณ์นี้ในเมื่อจบการแปล

ดังนั้นจากโค้ดข้างบนจะเกิดเหตุการณ์ดังนี้

	→	เริ่มต้นเอกสาร
<name>	→	อีลีเมนต์เริ่มต้น name
<first>	→	อีลีเมนต์เริ่มต้น first
Jim	→	ตัวอักษร Jim
</first>	→	อีลีเมนต์สิ้นสุด first
<last>	→	อีลีเมนต์เริ่มต้น last
Jones	→	ตัวอักษร Jones
</last>	→	อีลีเมนต์สิ้นสุด last
</name>	→	อีลีเมนต์สิ้นสุด name
	→	สิ้นสุดเอกสาร

ทั้ง 2 วิธีมีข้อดีข้อเสียต่างกันคือ DOM จะใช้เมโมรีและมีการประมวลผลมาก แต่สามารถจัดการได้ง่าย สามารถที่จะแก้ไขค่าได้ ไม่เหมาะกับเอกสาร XML ที่มีขนาดใหญ่ ถูกลำบากใช้ในเบราว์เซอร์ ในขณะที่ SAX ไม่ต้องแปลเอกสาร XML ทั้งหมดจึงใช้เมโมรีและมีการประมวลผลน้อยกว่า เหมาะกับการสร้างและรับค่าจากเอกสาร จะถูกใช้ในการประมวลผลทางฝั่งเซิร์ฟเวอร์ ตัวอย่าง XML พาร์เซอร์ดังตารางที่ 2-4

ชื่อ	แพลตฟอร์ม	คำบรรยาย
ActiveDOM	Microsoft Window	http://www.vivid-creations.com
ActiveSAX	Microsoft Window	http://www.vivid-creations.com
JavaTM API for XML	Java	http://java.sun.com/xml
MS XML	Microsoft Window	http://www.microsoft.com
Oracle XML Developer's Kit	Java , C++ , PL/SQL	http://technet.oracle.com/tech/xml
Xerces	Java , C++	http://xml.apache.org
XP	Java	http://www.jclark.com/xml

ตารางที่ 2-4 XML พาร์เซอร์

2.1.10 ประโยชน์ของ XML

สำหรับประโยชน์ของ XML นั้น เป็นด้านความยืดหยุ่นในการใช้งานสำหรับแอปพลิเคชันที่อิงกับเว็บที่ง่ายในการค้นหาข้อมูล มีความยืดหยุ่นในการพัฒนาเว็บ สามารถผสมผสานข้อมูลจากหลายแหล่งจากแอปพลิเคชันที่ต่างกัน สามารถแสดงข้อมูลแบบต่างๆ และสามารถปรับปรุงข้อมูลให้ทันสมัยเสมอ และคาดว่าจะเป็มาตรฐานใหม่ของระบบเปิด ซึ่งนับเป็นรูปแบบใหม่สำหรับการส่งข้อมูลบนเว็บที่มากด้วยข้อมูลหลายแบบ แต่ส่งผ่านด้วยเทคโนโลยีที่บีบอัดข้อมูลที่ให้ความเร็วได้รับการสนับสนุนจากผลิตภัณฑ์ค่ายไมโครซอฟท์

สถาปัตยกรรม XML

ภาษา XML ได้รับการสนับสนุนจาก W3C ให้นักพัฒนาเว็บแอปพลิเคชันได้หันมาใช้เป็นส่วนประกอบของการพัฒนาเว็บไซต์ เพราะ XML มีประสิทธิภาพและมีความน่าเชื่อถือสูงในการแปลงข้อมูลและโครงสร้างข้อมูลให้สามารถนำไปใช้งาน สถาปัตยกรรม 3 เทียร์ ที่ XML สามารถสร้างขึ้นจากระบบข้อมูลที่ใช้โมเดลของ 3-เทียร์ โครงสร้าง ของข้อมูลต่างๆ สามารถนำมาแสดงตามข้อกำหนด หรือรูปแบบที่ต้องการตามการใช้งานได้

XML เป็นการทำงานในระดับกลาง มิดเดิลเทียร์ที่สามารถเรียกใช้ฐานข้อมูลได้หลากหลายระบบ ฐานข้อมูลและโอนข้อมูลให้อยู่ในรูปแบบของ XML และมีการให้รายละเอียดเกี่ยวกับตัวข้อมูล โครงสร้างต่างๆ ของระบบฐานข้อมูลได้ XML เป็นระบบเปิดที่นำเสนอข้อมูลในรูปแบบเท็กซ์ผ่านทาง HTTP เหมือนกับ HTML แต่จะมีคุณสมบัติในการให้ข้อมูลแบบตามความเป็นจริง (real time) อัปเดตหรือเปลี่ยน

แปลงได้ตามข้อกำหนด การแสดงข้อมูลจาก XML ใน HTML จะเป็นการเพิ่มในส่วนของรายละเอียดข้อมูล ที่มีการเรียกใช้จากแหล่งหรือฐานข้อมูลที่เชื่อมโยงกันในหลายแหล่งเพื่อให้ HTML มีความสมบูรณ์มากขึ้น ในอนาคตการพัฒนาเว็บหรือการเขียน และสร้าง HTML ไม่จำเป็นต้องมีการเขียนชุดคำสั่งที่ยุ่งยากซับซ้อนมากก็สามารถทำงานร่วมกับระบบข้อมูลได้อย่างมีประสิทธิภาพ XML จะทำการกำหนดค่าสำหรับโครงสร้างข้อมูลที่จะนำไปแสดงใน HTML นอกจากนั้นยังสามารถนำไปสนับสนุนระบบการแลกเปลี่ยนข้อมูลข่าวสารทางอิเล็กทรอนิกส์ได้อย่างดีอีกด้วย

2.1.11 บทสรุปของ XML

XML มีความยืดหยุ่นพอในการนำเสนอข้อมูลแบบต่างๆ ที่สามารถให้รายละเอียดโครงสร้างข้อมูลตามระดับและความต้องการในการนำไปใช้งาน

2.2 SOAP

2.2.1 แนะนำ SOAP

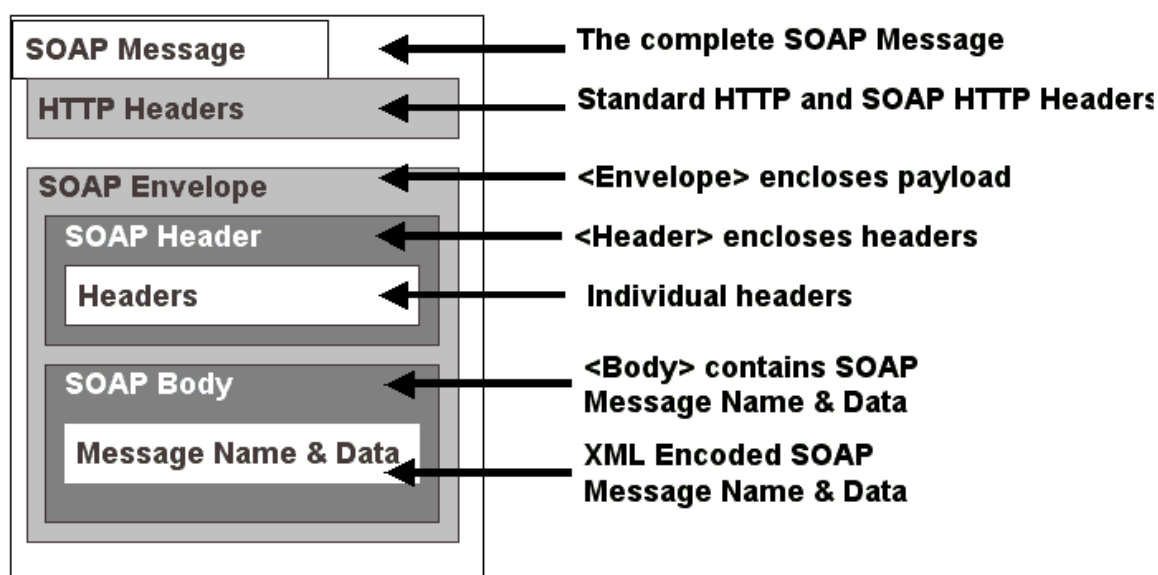
SOAP (Simple Object Access Protocol) เป็นโพรโทคอลที่ใช้ XML เป็นพื้นฐาน เพื่อให้ซอฟต์แวร์คอมพิวเตอร์ และแอปพลิเคชันสามารถติดต่อกันผ่าน HTTP ซึ่งเป็นมาตรฐานอินเทอร์เน็ตโพรโทคอลได้

เนื่องจากการสื่อสารระหว่างแอปพลิเคชันจำเป็นต้องการพัฒนาแอปพลิเคชันบนอินเทอร์เน็ต เป็นอย่างมาก แต่แอปพลิเคชันแบบกระจายกันทำงาน (distributed application) ในปัจจุบันใช้ Remote Procedure Call (RPC) สื่อสารกันระหว่างออบเจกต์ เช่น DCOM และ CORBA ซึ่งไม่ได้ใช้พอร์ตเดียวกับ HTTP ที่เป็นพอร์ตมาตรฐานสำหรับให้บริการเว็บ ดังนั้น RPC จึงนำมาปรับใช้กับอินเทอร์เน็ตได้ยาก และมีปัญหาทางด้านความปลอดภัย ไฟร์วอลล์และพร็อกซีเซิร์ฟเวอร์จะไม่ยอมให้ส่งข้อมูลชนิดนี้ได้ตามปกติวิธีที่ดีกว่าคือใช้ HTTP เพราะเป็นที่ยอมรับโดยอินเทอร์เน็ตบราวเซอร์ และเซิร์ฟเวอร์ทุกชนิด ซึ่ง SOAP ถูกสร้างมาเพื่อใช้ในกรณีนี้

ข้อดีของ SOAP ก็คือ SOAP ไม่ขึ้นกับคอมพิวเตอร์เทคโนโลยี และภาษาการเขียนโปรแกรมใดๆ สามารถเขียนได้ง่ายและขยายเพิ่มเติมได้

2.2.2 ส่วนประกอบของ SOAP

SOAP เมสเสจใช้ไวยากรณ์ของ XML ในการสร้าง ประกอบด้วย 3 อีลีเมนต์มาตรฐาน คือ SOAP Envelope, SOAP Header และ SOAP Body



รูปที่ 2-3 โครงสร้างของ SOAP

SOAP เมสเสจ จะต้องเป็นไปตามกฎนี้

- Envelope เป็นอีลีเมนต์ที่อยู่บนสุด ต้องมีอีลีเมนต์นี้เสมอ
- Header อาจจะมีหรือไม่มีก็ได้ แต่ถ้ามีต้องเป็นอีลีเมนต์ลูกอีลีเมนต์แรกของ envelope
- Body ต้องเป็นอีลีเมนต์ลูกอีลีเมนต์แรกของ envelope หรือ header

ตัวอย่างของ SOAP เมสเสจ ตัวอย่างนี้คือ GetLastTradePrice SOAP ร้องขอ (request) ที่ส่งไปยัง StockQuote เซอร์วิสประกอบด้วยสตริงพารามิเตอร์และคืนค่าโฟลท (float) กลับมากับ SOAP ตอบสนอง SOAP Envelope อีลีเมนต์จะอยู่ที่จุดบนสุดของเอกสาร XML XML เนมสเปซใช้เพื่อป้องกันการสับสนของ SOAP identifier

SOAP ร้องขอจะเป็นดังนี้

```
POST /StockQuote HTTP/1.1
```

```
Host: www.stockquoteserver.com
```

```
Content-Type: text/xml; charset="utf-8"
```

```
Content-Length: nnnn
```

```
SOAPAction: "Some-URI"
```

```
<SOAP-ENV:Envelope
```

```
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
```

```
  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
```

```
  <SOAP-ENV:Body>
```

```
    <m:GetLastTradePrice xmlns:m="Some-URI">
```

```

    <symbol>DIS</symbol>

  </m:GetLastTradePrice>

</SOAP-ENV:Body>

</SOAP-ENV:Envelope>

```

SOAP ตอบสนองจะเป็นดังนี้

```

HTTP/1.1 200 OK
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn

<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:GetLastTradePriceResponse xmlns:m="Some-URI">
      <Price>34.5</Price>
    </m:GetLastTradePriceResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

นอกจากนั้นใน SOAP เมสเสจ จะมีการใช้ XML เนมสเปซ ทุกๆ อีลีเมนต์ในเอกสารจะขึ้นต้นด้วยเนมสเปซ เนมสเปซจะถูกกำหนดโดยใช้ xmlns แอดทริบิวต์ดังนี้

```

<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">

```

แต่หากไม่ต้องการให้ขึ้นต้นด้วยเนมสเปซก็สามารถทำได้ ดังนี้

```

<Envelope
  xmlns="http://schemas.xmlsoap.org/soap/envelope/">

```

2.2.2.1 Envelope

envelope อีลีเมนต์เป็นอีลีเมนต์บนสุดของเอกสาร XML ที่ใช้แสดงเมสเสจ อาจประกอบด้วย แอตทริบิวต์ คือ envelope เนมสเปซ และ encodingStyle

1. envelope เนมสเปซ – SOAP เมสเสจจะแสดงเวอร์ชันโดยใช้ เนมสเปซ ของ envelope อีลีเมนต์เนมสเปซ จะถูกนำไปใช้ขึ้นต้น envelope, header และ Body อีลีเมนต์
2. EncodingStyle แอตทริบิวต์ – ใช้แสดงวิธี serialization ของ SOAP เมสเสจ แอตทริบิวต์นี้สามารถมีได้ในทุกอีลีเมนต์และจะมีผลกับ คอนเทนต์ของอีลีเมนต์นั้นและอีลีเมนต์ลูกทั้งหมดของมันที่ไม่ได้ประกาศแอตทริบิวต์

2.2.2.2 Body

เป็นส่วนของข้อมูลที่ใช้ในการแลกเปลี่ยน โดยทั่ว ๆ ไปข้อมูลจะเป็นการ marshall RPC call และ error report อีลีเมนต์ลูกของBody อีลีเมนต์จะถูกเรียกว่า Body entry

Body entry จะต้องเป็นไปตามกฎนี้

- Body entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย เนมสเปซ URI และ ชื่อของมัน (local name)
- SOAP encodingStyle แอตทริบิวต์อาจจะมีได้ เพื่อแสดงถึงวิธีเข้ารหัส (encode) ของ Body entry นั้น

2.2.2.3 header (Header)

เป็นส่วนเพิ่มเติมของเมสเสจสามารถประกอบด้วยอินฟอร์เมชันที่ระบุไปยังแอปพลิเคชัน ส่วนเพิ่มเติมนี้อาจนำไปใช้โพลิเมอริคเป็น authentication, transaction management เป็นต้น ทุก ๆ อีลีเมนต์ลูกของ header อีลีเมนต์จะถูกเรียกว่า Header entry

Header entry จะต้องเป็นไปตามกฎดังนี้

- Header entry จะต้องแสดงด้วยชื่อเต็มประกอบด้วย เนมสเปซ URI และ ชื่อของมัน (local name)
- อาจมี SOAP encodingStyle แอตทริบิวต์ที่ใช้สำหรับHeader entry
- SOAP mustUnderStand แอตทริบิวต์และ SOAP actor แอตทริบิวต์จะมีหรือไม่ก็ได้ เพื่อใช้บอกว่าจะจัดการกับเอ็นทรีนั้นอย่างไรและโดยใคร

```
<soap:Header>
```

```
<m:local xmlns:m="http://www.w3schools.com/local/"
```

```
soap:actor="http://www.w3schools.com/appml" />
```

```
<m:language>en</m:language>
```

```
<m:currency>USD</m:currency>
```

```
</m:local>
</soap:Header>
```

- 1) SOAP Actor แอดทริบิวต์ – SOAP เมสเสจสามารถถูกส่งไปยังปลายทางผ่านตัวกลางของ SOAP (SOAP intermediary) ตัวกลางของ SOAP คือแอปพลิเคชันที่มีความสามารถทั้งรับและส่งต่อ SOAP เมสเสจ ในการส่งต่อตัวกลางจะต้องไม่ส่งต่อเฮดเดอร์อิลีเมนต์ไปให้กับแอปพลิเคชัน โดยตัวกลางนี้จะถูกกำหนดเป็น URI ถ้าไม่มี แอดทริบิวต์นี้หมายถึงผู้รับ SOAP เมสเสจเป็นปลายทางสุดท้าย
- 2) SOAP MustUnderStand แอดทริบิวต์ – ใช้แสดงว่า header entry นี้จำเป็นหรือเป็นเพียงอ็อปชันสำหรับผู้รับที่จะนำไปประมวลผล ค่าของมันคือ “1” หรือ “0” ถ้าไม่มีการระบุจะมีค่าเท่ากับ “0” โดย “0” หมายถึงเป็นอ็อปชัน และ “1” หมายถึงจำเป็น โดยจะต้องประมวลผลให้ถูกต้องตามกฎ (semantic) หรือต้องผิดพลาด (fail) เช่น เมสเสจเป็นส่วนหนึ่งของทรานแซกชัน ถ้าปลายทางไม่รองรับทรานแซกชันจะต้องไม่ประมวลผลและส่งผลผิดพลาดกลับไปในกรณีนี้ mustUnderStand แอดทริบิวต์เป็น “1” แต่ถ้าเป็น “0” จะยังคงประมวลผลต่อไป

2.2.3 SOAP ฟอลต์อิลีเมนต์ (SOAP Fault Element)

ข้อความแสดงความผิดพลาดจากแอปพลิเคชันของ SOAP จะเก็บอยู่ในฟอลต์อิลีเมนต์ ซึ่งถ้ามีจะต้องปรากฏใน body อิลีเมนต์เพียงครั้งเดียวใน SOAP เมสเสจ ตัวอย่างอาจเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:MustUnderstand</faultcode>
      <faultstring>Mandatory Header error.</faultstring>
      <faultactor>http://www.wrox.com/heroes/endpoint.asp</faultactor>
      <detail>
        <w:source xmlns:w="http://www.wrox.com/">
          <module>endpoint.asp</module>
          <line>203</line>
        </w:source>
      </detail>
    </soap:Fault>
```

```
</soap:Body>
</soap:Envelope>
```

SOAP ฟอลต์อีลีเมนต์มีอีลีเมนต์ย่อย ๆ ดังตารางที่ 2-5

Sub Element	Description
<faultcode>	โค้ดที่ระบุถึงการ error
<faultstring>	ข้อความการ error
<faultactor>	ใครเป็นสาเหตุของการ error
<detail>	ระบุอินฟอร์เมชันของการ error

ตารางที่ 2-5 SOAP Fault Element

ค่าของฟอลต์โค้ด (faultcode) สามารถมีค่าได้ดังตารางที่ 2-6

Error	Description
VersionMismatch	เนมสเปซ ภายใน SOAP เอ็นโวลอปอีลีเมนต์ไม่ถูกต้อง
MustUnderstand	อีลีเมนต์ลูกของเซดเคอร์อีลีเมนต์กับ mustUnderstand แอตทริบิวต์ที่มีค่า "1" ผู้รับไม่รองรับ
Client	เมสเซจมีรูปแบบไม่ถูกต้อง หรือมีอินฟอร์เมชันที่ไม่ถูกต้อง
Server	เกิดปัญหากับเซิร์ฟเวอร์ ไม่สามารถประมวลผลได้

ตารางที่ 2-6 ค่าฟอลต์โค้ด

2.2.4 SOAP เอ็นโวลอป

SOAP เอ็นโวลอปมีวิธีการ map จากไทป์ของโปรแกรมมิ่งไปเป็น XML 2 วิธี คือจากภายนอก โดยใช้ WSDL (Web Services Description Language) ที่บอกถึงไทป์ของข้อมูลที่ได้รับหรือส่ง หรือใช้ xsi:type แอตทริบิวต์ในกรณีที่ใช้ภาษาที่ไม่รองรับ WSDL โดยทั้ง 2 วิธีจะใช้โครงสร้างของ XML ในการระบุไทป์ ตัวอย่างของการใช้ xsi จะเป็นดังนี้

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance">
```



```

xmlns:xsd="http://www.w3.org/1999/XMLSchema">
<soap:Body>
  <m:MixedMessage xmlns:m="http://www.wrox.com/mix/">
    <param1 xsi:type="xsd:string">OU812</param1>
    <param2 xsi:type="xsd:integer">2001</param2>
    <param3 xsi:type="xsd:double">3.14159</param3>
  </m:MixedMessage>
</soap:Body>
</soap:Envelope>

```

SOAP รองรับไทป์ของข้อมูลได้ทั้งไทป์พื้นฐาน (simple type) และไทป์ประกอบ (compound type) เช่นสตรัค (struct) และอาร์เรย์ (array)

2.2.4.1 ตัวอย่างสตรัค (Struct)

สตรัค book ที่เขียนด้วยภาษา c++ จะเป็นดังนี้

```

struct Book
{
  string author;
  string name;
  int page;
};

```

Soap เมสเสจที่ไม่ใช่ WSDL แต่ใช้ xsi จะเป็นดังนี้

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/1999/XMLSchema">
<SOAP-ENV:Body>
  <ns1:searchBook xmlns:ns1="http://soapinterop.org/" SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<bookResult xmlns:ns2="http://soapinterop.org/xsd" xsi:type="ns2:Book">
  <author xsi:type="xsd:string">Ed Roman</author>

```

```

<name xsi:type="xsd:string">Mastering EJB</intro>
<page xsi:type="xsd:int">500</page>
</bookResult>
</ns1:searchBook>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศโครงสร้างจะต้องประกอบด้วยโครงสร้างดังนี้

```

<types>
  <schema target namespace="http://soapinterop.org/xsd"
    xmlns="http://www.w3.org/2001/XMLSchema"
    xmlns:SOAP-ENC="http://schemas.xmlsoap.org/soap/encoding/"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
    elementFormDefault="qualified">
    <complexType name="Book">
      <sequence>
        <element name="author" type="string"/>
        <element name="name" type="string"/>
        <element name="page" type="int"/>
      </sequence>
    </complexType>
  </schema>
</types>

```

2.2.4.2 ตัวอย่างอาร์เรย์

ตัวอย่างนี้เป็นอาร์เรย์ของสตริง book ในตัวอย่างก่อน Soap เมสเซจที่ไม่ใช้ WSDL แต่ใช้ xsi จะเป็นดังนี้

```

<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <SOAP-ENV:Body>

```

```

<ns1:BookArrayResult xmlns:ns1="http://soapinterop.org/" SOAP-
ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <BookArray xmlns:ns2="http://schemas.xmlsoap.org/soap/encoding/"
    xsi:type="ns2:Array" xmlns:ns3="http://soapinterop.org/xsd"
    ns2:arrayType="ns3:Book[3]">
    <item xsi:type="ns3:Book">
      <author xsi:type="xsd:string">Ed Roman</author>
      <name xsi:type="xsd:string">Mastering EJB</name>
      <page xsi:type="xsd:int">500</page>
    </item>
    <item xsi:type="ns3:Book">
      <author xsi:type="xsd:string">Roger Session</author>
      <name xsi:type="xsd:string">The Battle of the  middleware</name>
      <page xsi:type="xsd:int">350</page>
    </item>
    <item xsi:type="ns3:Book">
      <author xsi:type="xsd:string">Chris Dix</author>
      <name xsi:type="xsd:string">Professional XML Web Service</name>
      <page xsi:type="xsd:int">550</page>
    </item>
  </BookArray>
</ns1:BookArrayResult>
</SOAP-ENV:Body>
</SOAP-ENV:Envelope>

```

แต่ถ้าใช้ WSDL ในส่วนของการประกาศโครงสร้างจะต้องประกอบด้วยโครงสร้างดังนี้

```

<types>
  <schema target namespace='http://soapinterop.org/xsd'
    xmlns='http://www.w3.org/2001/XMLSchema'
    xmlns:SOAP-ENC='http://schemas.xmlsoap.org/soap/encoding'
    xmlns:wSDL='http://schemas.xmlsoap.org/wSDL/'
    elementFormDefault='qualified'>

```

```

<complexType name="Book">
  <sequence>
    <element name='author' type='string'/>
    <element name='name' type='string'/>
    <element name='page' type='int'/>
  </sequence>
</complexType>
<complexType name='ArrayOfBook'>
  <complexContent>
    <restriction base='SOAP-ENC:Array'>
      <attribute ref='SOAP-ENC:arrayType' wsdl:arrayType='typens:Book[]'/>
    </restriction>
  </complexContent>
</complexType>
</schema>
</types>

```

2.2.5 SOAP ใน HTTP

ในการส่ง SOAP ผ่านทาง HTTP จะต้องใช้ content-type เป็น text/xml แต่ใน SOAP ร้องขอจะต้องมีเฮดเดอร์ SOAP Action ภายใน HTTP เฮดเดอร์ SOAP Action จะเป็นตัวบอกให้เซิร์ฟเวอร์รู้ว่า HTTP Post นั้นเป็น SOAP เมสเสจ และค่าของ เฮดเดอร์ คือ URI ที่แสดงถึงจุดหมายของ SOAP เมสเสจ ส่วน SOAP ตอบสนองจะต้องมี status code ตามมาตรฐานของ HTTP โดย 200-299 แสดงว่าสำเร็จ แต่ถ้าตอบสนองเมสเสจ เป็นการฟอลต์ แล้ว status code จะต้องเป็น 500 ซึ่งแสดงถึง internal server error ตัวอย่างของตอบสนองที่มี status code เป็น 500 อาจเป็นดังนี้

```

HTTP/1.1 500 Internal Server Error
Content-Type: text/xml
Content-Length: ###
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
  xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/1999/XMLSchema">
  <soap:Body>

```

```

<soap:Fault>
  <faultcode>soap:VersionMismatch</faultcode>
  <faultstring>The SOAP เนมสเปซ is incorrect.</faultstring>
  <faultactor>http://www.wrox.com/endpoint.asp</faultactor>
  <detail>
    <w:errorinfo xmlns:w="http://www.wrox.com/">
      <desc>The SOAP เนมสเปซ was blank.</desc>
    </w:errorinfo>
  </detail>
</soap:Fault>
</soap:Body>
</soap:Envelope>

```

2.2.6 SOAP สำหรับ RPC

จุดประสงค์ของการออกแบบ SOAP คือทำ RPC โดยใช้ XML การเรียก RPC นั้นจะ map เข้ากับ HTTP ร้องขอ และ RPC ตอบสนองจะ map กับ HTTP ตอบสนอง

ในการทำ mehod call จำเป็นที่จะต้องมียีนฟอร์เมชันดังนี้

1. URI ของออบเจ็กต์ปลายทาง (target object)
2. ชื่อของเมธอด
3. คำอธิบายของเมธอด (method signature) ส่วนนี้เป็นอ็อปชัน
4. พารามิเตอร์สำหรับเมธอด
5. ข้อมูลส่วนหัว (header data) เป็นอ็อปชัน

พิจารณาเมธอดดังนี้

```
double GetStockQuote ( [in] stirng sSymbol);
```

ถ้าเนมสเปซของเมธอดคือ “http://www.worxstox.com/” แล้วการเรียกเมธอดโดยร้องขอ stock quote ใช้สัญลักษณ์ OU812 จะเป็นดังนี้

```

<q:GetStockQuote xmlns:q="http://www.wroxstox.com/">
  <q:sSymbol xsi:type="xsd:string">OU812</q:sSymbol>
</q:GetStockQuote>

```

ชื่อเมธอดและชื่อของอีลีเมนต์จะต้อง match กันเช่นเดียวกับพารามิเตอร์

ถ้าห้รับ response จะต้องชื่ออีลีเมนต์จะต้องเป็นชื่อเมธอดตามด้วยคอบสนอง ดังนี้

```
<q:GetStockQuoteResponse xmlns:q="http://www.wroxstox.com/"
  <q:ret xsi:type="xsd:double">100.0</q:ret>
</q:GetStockQuoteResponse>
```

ชื่ออีลีเมนต์ของค่าที่คืนมาสามารถเปลี่ยนได้ โดยจะกำหนดได้จากโปรแกรมที่เขียนหรือจาก WSDL

2.2.7 SOAP Toolkit

SOAP toolkit คือเครื่องมือที่จะทำหน้าที่ในการประมวลผลการร้องขอ SOAP และส่งการตอบสนองกลับไปให้ไคลเอนต์ โดย SOAP toolkit แต่ละตัวใช้ได้บนแพลตฟอร์มที่ต่างกัน รองรับเทคโนโลยีเว็บเซอร์วิสต่างกัน บางตัวอาจจะรองรับ WSDL หรือ UDDI ในขณะที่บางตัวรองรับอย่างใดอย่างหนึ่งหรือไม่รองรับทั้งสองอย่าง และถึงแม้ว่า SOAP จะเป็นอิสระต่อแพลตฟอร์ม แต่ใน SOAP toolkit บางตัวยังไม่สามารถทำงานข้ามผลิตภัณฑ์ (interoperability) ได้ เนื่องจากรองรับเทคโนโลยีของเว็บเซอร์วิส ไม่เหมือนกัน เช่น xsi ในบางผลิตภัณฑ์จะบังคับให้ SOAP เมสเสจจะต้องระบุไทป์ด้วย xsi หากไม่มีก็จะความผิดพลาดซึ่งในปัจจุบันแต่ละผลิตภัณฑ์ก็พยายามพัฒนาให้สามารถทำงานด้วยกันได้

ชื่อ	แพลตฟอร์ม	แหล่งที่มา
4s4c	COM	http://www.4s4c.com
A SOAP for RPC NT Service	COM	http://www.whitemesa.com
Apache Axis	Java	http://xml.apache.org/axis
Apache SOAP	Java	http://xml.apache.org/soap
CapeConnect	Java	http://www.capeclear.com
Glue	Java	http://www.themindelectric.com
IBM Web Services Toolkit	Java	http://www.alphaworks.ibm.com
IdooXoap for Java and C++	Java , C++	http://www.zvon.org
Microsoft SOAP Toolkit	COM	http://www.microsoft.com
PocketSOAP	COM	http://www.pocketsoap.com

SOAP::Lite	Perl	http://www.soaplite.com
Visual Studio .NET	COM	http://www.microsoft.com

ตารางที่ 2-7 SOAP Toolkit

2.3 UDDI

UDDI (Universal Description, Discovery and Integration) เป็นมาตรฐานที่จัดตั้งขึ้น โดยบริษัท ไอบีเอ็ม, ไมโครซอฟท์และบริษัทยักษ์ใหญ่ทางธุรกิจ B2B (Business-to-Business) อื่น ๆ UDDI ถูกสร้างขึ้นมาเป็นมาตรฐาน ในการค้นหาบริการเว็บเซอร์วิสสำหรับคู่ค้าทางธุรกิจ ซึ่งเปรียบได้กับฐานข้อมูลขนาดใหญ่ ซึ่งมีข้อมูลของเว็บเซอร์วิสที่เปิดให้บริการ โดยที่เว็บไซต์สำหรับค้นหาเว็บเซอร์วิสมีอยู่หลายตัวอย่างเช่น

- <http://uddi.microsoft.com/search.aspx>
- <http://www-3.ibm.com/services/uddi/testregistry/find>
- <http://uddi.org>

2.4 WSDL

WSDL (Web Services Description Language) เกิดจากความร่วมมือระหว่างบริษัทไอบีเอ็มและไมโครซอฟท์ WSDL เป็นภาษาที่ใช้อธิบายคุณลักษณะของเว็บเซอร์วิสและวิธีการติดต่อกับเว็บเซอร์วิส นั้น ๆ โดยใช้ไวยากรณ์ของภาษา XML ซึ่ง WSDL อยู่ในความดูแลของ W3C เวอร์ชันล่าสุด คือ WSDL 1.1 รายละเอียด สามารถหาอ่านเพิ่มเติมได้จากเว็บไซต์ของ W3C ที่ <http://www.w3.org/TR/wsdl> ส่วนในทางปฏิบัติ หากเราต้องการ สร้างเว็บเซอร์วิสขึ้นมาเป็นของตนเอง ก็สามารถสร้างเอกสาร WSDL ได้โดยอัตโนมัติ เราจึงไม่ต้องไปกังวลในรายละเอียด ในข้อกำหนดใน WSDL มากนักโดยตัวอย่างเอกสาร WSDL เช่น

```
<?xml version="1.0"?>
```

```
<definitions name="StockQuote"
```

```
target namespace="http://example.com/stockquote.wsdl"
```

```
xmlns:tns="http://example.com/stockquote.wsdl"
```

```
xmlns:xsd1="http://example.com/stockquote.xsd"
```

```
xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
```

```
xmlns="http://schemas.xmlsoap.org/wsdl/">
```

```
<types>
```

```
<schema target namespace="http://example.com/stockquote.xsd"
```

```

    xmlns="http://www.w3.org/2000/10/XMLSchema">
<element name="TradePriceRequest">
  <complexType>
    <all>
      <element name="tickerSymbol" type="string"/>
    </all>
  </complexType>
</element>
<element name="TradePrice">
  <complexType>
    <all>
      <element name="price" type="float"/>
    </all>
  </complexType>
</element>
</schema>
</types>

<message name="GetLastTradePriceInput">
  <part name="body" element="xsd1:TradePriceRequest"/>
</message>

<message name="GetLastTradePriceOutput">
  <part name="body" element="xsd1:TradePrice"/>
</message>

<portType name="StockQuotePortType">
  <operation name="GetLastTradePrice">
    <input message="tns:GetLastTradePriceInput"/>
    <output message="tns:GetLastTradePriceOutput"/>
  </operation>
</portType>

```



```

<binding name="StockQuoteSoapBinding" type="tns:StockQuotePortType">
  <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
  <operation name="GetLastTradePrice">
    <soap:operation soapAction="http://example.com/GetLastTradePrice"/>
    <input>
      <soap:body use="literal"/>
    </input>
    <output>
      <soap:body use="literal"/>
    </output>
  </operation>
</binding>

<service name="StockQuoteService">
  <documentation>My first service</documentation>
  <port name="StockQuotePort" binding="tns:StockQuoteBinding">
    <soap:address location="http://example.com/stockquote"/>
  </port>
</service>
</definitions>

```